Student-to-Tutor Scheduling and Availability Management System

TECHNICAL & USER MANUAL

Detailed Final Version

Project Type	Individual project
Student	Nicolas Imhof
Student ID	700864476
Email	nimhof@oldwestbury.edu
Submitted To	Dr. Renu Balyan
Final Website URL	https://student-tutor-scheduling-system.pages.dev

Disclaimer: This document is prepared for course submission and system evaluation only. The system is an academic project and not intended for production academic advising or institutional scheduling without further review, security hardening, and formal testing.

TABLE OF CONTENTS

1. Introduction

1.1 Project Overview

The Student-to-Tutor Scheduling and Availability Management System is a web-based scheduling application designed to connect students with available tutors and give administrators control over tutor approvals, time-off requests, system events, and review management. The final hosted website is https://student-tutor-scheduling-system.pages.dev.

The system was created as an individual academic project by Nicolas Imhof. It reflects college-level programming knowledge in HTML, CSS, JavaScript, SQL, and basic package-based web development. The project does not attempt to use an unnecessarily complex framework; instead, it uses simpler and understandable methods that match the course level while still creating a functional multi-role application.

The system is meant to solve a realistic school scheduling problem: students need a simple way to find tutors, tutors need a way to show availability and request time off, and administrators need oversight. The project includes a login page, registration page, role-based dashboard, appointment scheduling workflow, reviews workflow, user management tools, and calendar/system event features.

1.2 Document Overview

This manual acts as both a technical reference and a user guide. It explains how the system is built, why certain design choices were made, how to download and install required software, how to run the project locally, how each page is meant to function, and how each role uses the system.

Sections 1 and 2 explain the purpose, architecture, software requirements, database structure, API/RPC logic, and design reasoning. Section 3 explains the user-facing features page by page. Section 4 explains the local installation and execution process in step-by-step detail. Sections 5 and 6 describe performance, current limitations, future improvements, troubleshooting, and support.

1.3 Development Context and AI Tool Use

This project was completed individually by Nicolas Imhof. Trae AI was used as the coding software/editor, similar to how another developer might use VS Code. Trae AI was not treated as the project owner; it was used as a coding assistant inside the development process.

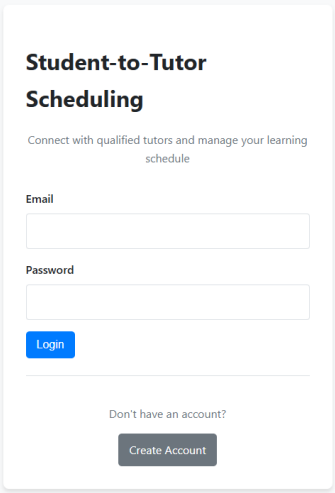
The AI within Trae was used for three main purposes: first, to generate realistic seed data for the Supabase database so the system could be tested with students, tutors, admins, courses, appointments, and reviews; second, to help fix more complicated errors in the code that were difficult to solve alone; and third, to streamline the process of preparing and uploading the project files to GitHub when the repository was being set up.

The coding approach was still based largely on college-level web programming knowledge. The project uses direct HTML, CSS, JavaScript, and SQL rather than a heavy framework. This made the code easier to read, easier to debug, and easier to explain in a technical manual.

1.4 System Overview

The application has two major states: a logged-out state and a logged-in dashboard state. The logged-out state shows either the login page or registration page. After authentication succeeds and the account is approved, script.js renders the dashboard based on the logged-in user role.

The browser loads the static front-end files from Cloudflare Pages. JavaScript connects to Supabase using the Supabase JavaScript client. Supabase stores the PostgreSQL database, authentication records, tables, views, and database functions. The front end calls Supabase directly for login, data retrieval, appointment booking, review updates, event creation, and user management.



2. System Description

2.1 Requirements and Full Download Process

2.1.1 Requirements for End Users

End users only need a modern web browser and an internet connection. Chrome, Edge, or Firefox are recommended. Users do not need to install Trae AI, Node.js, npm, Git, or Supabase tools just to use the deployed website.

- Operating system: Windows, macOS, or another system capable of running a modern browser.
- Browser: Chrome, Edge, or Firefox recommended.
- Internet connection: Required because the hosted website, Supabase authentication, and Supabase database are cloud-based.
- Account approval: New Student and Tutor accounts must be approved before they can access their dashboards.

2.1.2 Requirements for Developers and Local Installation

Developers or evaluators who want to run the project locally need several tools. The important point is that coding languages and packages are not installed inside Trae AI itself. Trae AI is the editor. Node.js, npm, Git, and the project packages are installed on the computer or inside the project folder, then Trae uses them through its integrated terminal.

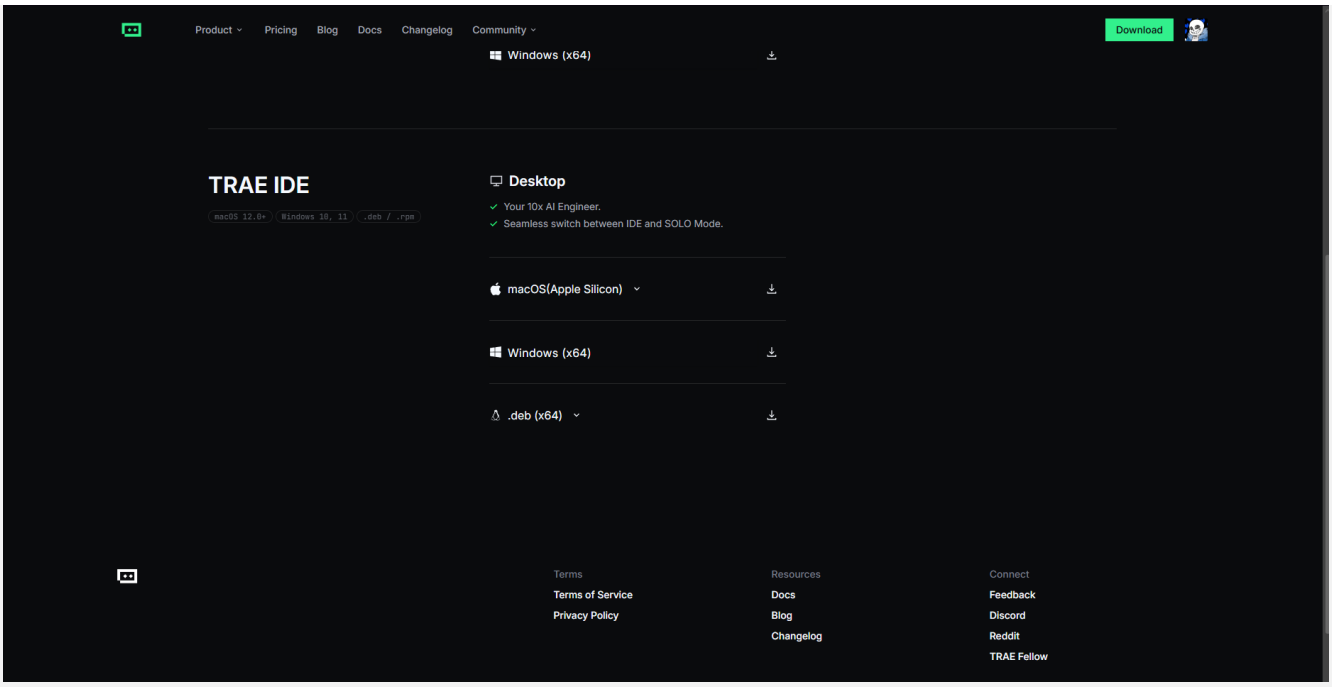
Item	Required?	Purpose
Trae AI IDE	Yes for this documented workflow	Coding software/editor used for opening, editing, debugging, and running terminal commands for the project.
Node.js LTS + npm	Yes	Node runs JavaScript tooling; npm installs project packages and runs http-server.
Git	Recommended	Used to initialize the repository, commit code, and upload changes to GitHub.

GitHub account	Recommended for deployment	Stores the repository that Cloudflare Pages deploys from.
Supabase account/project	Yes for database/auth	Hosts the PostgreSQL database, authentication, SQL functions, and seeded data.
Cloudflare account	Yes for web hosting	Hosts the final website on the pages.dev URL.
Modern browser	Yes	Used to test login, dashboard, and every role-based feature.

A. Downloading and Installing Trae AI IDE

Use this process on Windows 10 or Windows 11. The official Trae download page lists Windows x64 as an available desktop download for Trae IDE.

1. Open a browser and go to https://www.trae.ai/download.
2. Find the TRAE IDE section. Do not confuse it with TRAE SOLO unless you specifically intend to use SOLO. For this project, the IDE/editor is the important tool.
3. Choose the Windows (x64) download option. Wait until the .exe installer finishes downloading.
4. Open the Downloads folder. Double-click the Trae AI installer file. If Windows asks for permission, click Yes only if the file came from the official Trae site.
5. Follow the installer prompts. Use the default installation location unless your computer requires another location.
6. When installation completes, launch Trae AI from the Start menu or desktop shortcut.
7. Sign in or create an account if Trae requests it. If sign-in is skipped, basic editor use may still work, but AI features may require a login.
8. In Trae, choose File > Open Folder and select the unzipped project folder: STUDENT-TO-TUTOR SCHEDULING AND AVAILABILITY MANAGEMENT SYSTEM.
9. Open the integrated terminal in Trae. Use Terminal > New Terminal or the keyboard shortcut Ctrl + ` . The terminal should open at the project folder. If it opens somewhere else, use cd to move into the project folder.



B. Downloading and Installing Node.js and npm

Node.js is needed because this project uses npm packages and npm scripts. npm is installed with Node.js. The official npm documentation explains that Node.js and npm must be installed before packages can be installed or run from the terminal.

10. Open https://nodejs.org in a browser.
11. Download the LTS version. LTS is recommended because it is the stable long-term support version used for most projects.
12. Run the Node.js installer. Leave the default options selected unless your computer has special restrictions.
13. Make sure the installer includes npm. This is normally selected by default.
14. Finish installation and close the installer.
15. Close and reopen Trae AI so the terminal can detect the newly installed node and npm commands.
16. Open Trae terminal and run node -v. If a version number appears, Node.js is installed correctly.
17. Run npm -v. If a version number appears, npm is installed correctly.
18. If either command says it is not recognized, restart the computer or reinstall Node.js using the LTS installer.

```
node -v
npm -v
```

C. Downloading and Installing Git

Git is needed if the project will be uploaded to GitHub through command-line repository steps. If using only GitHub web upload, Git is less necessary, but it is still recommended for version control.

19. Open https://git-scm.com/downloads.
20. Choose the Windows download and run the installer.
21. Accept the default options unless you understand a reason to change them.
22. After installation, close and reopen Trae AI.
23. Open Trae terminal and run git --version.
24. If a version number appears, Git is ready. If not, restart the computer or reinstall Git.

```
git --version
```

D. Downloading the Project ZIP and Opening It

25. Download the project zip file from the assignment submission package or course folder.
26. Right-click the zip file and choose Extract All. Do not run the project from inside the zipped folder.
27. Choose a simple location such as Documents or Desktop. Avoid a path with confusing special characters.
28. Open Trae AI, choose File > Open Folder, and select the extracted project folder.
29. Confirm that the project root contains index.html, register.html, script.js, calendar.js, style.css, calendar.css, package.json, package-lock.json, and the supabase folder.

E. Installing Project Packages in Trae Terminal

This project uses npm packages listed in package.json. The main packages are @supabase/supabase-js and http-server. The Supabase package lets JavaScript interact with Supabase; http-server runs the static files locally for testing.

30. Open the project in Trae AI.
31. Open Trae terminal with Terminal > New Terminal.
32. Make sure the terminal path is the project root. You should see files such as package.json when running dir on Windows.
33. Run npm install. This reads package.json/package-lock.json and downloads the exact packages into node_modules.
34. Wait until npm finishes. Do not close the terminal while packages are installing.
35. If npm gives an error, run npm cache verify, then run npm install again.
36. After installation, run npm start or npm run dev to start the local web server on port 8000.

```
dir
npm install
npm start
```

After npm start, open http://localhost:8000 in a browser. If port 8000 is already used, close the other server or adjust the port inside package.json.

F. Installing/Using Supabase JavaScript

The project uses the Supabase JavaScript client in two ways. In the browser, index.html and register.html load the CDN script. In the project package file, @supabase/supabase-js is also listed as an npm dependency so Node-based scripts can use the same library.

```
<script src="https://cdn.jsdelivr.net/npm/@supabase/supabase-js@2"></script>
npm install @supabase/supabase-js
```

2.2 Coding Languages, Packages, and Development Choices

Technology	Where Used	Reason for Use
HTML5	index.html and register.html	Defines the login page, registration page, dashboard containers, form fields, modal container, and linked scripts/styles. Chosen because it is a direct college-level web development foundation and makes the project easy to understand.
CSS3	style.css and calendar.css	Controls layout, cards, sidebar, buttons, forms, color themes, calendar grid, week view, month view, and responsive/mobile behavior. CSS variables were used so each role could have its own color theme.
JavaScript	script.js and calendar.js	Runs authentication, navigation, Supabase calls, form actions, calendar rendering, modals, search filtering, and role-based dashboard behavior. Vanilla JavaScript was used to keep the code understandable without requiring a framework.
SQL / PostgreSQL	Supabase SQL migrations and seed.sql	Creates tables, relationships, views, functions, and seed records. SQL was used because scheduling is relational: users, appointments, courses, reviews, events, and time-off requests depend on each other.
Node.js / npm	package.json and package-lock.json	Provides the package manager and local development server through http-server. It also installs the Supabase JavaScript package.
Supabase	Cloud database and authentication	Handles hosted PostgreSQL, authentication, account sessions, database functions, and application data. It reduced the need to build a separate custom backend server.
Cloudflare Pages	https://student-tutor-scheduling-system.pages.dev	Hosts the static front-end files from the GitHub repository. The free pages.dev domain was used instead of purchasing a custom domain.
Trae AI IDE	Coding environment	Used as the coding software/editor. Its AI features helped generate Supabase seed data, fix more complicated bugs, and streamline repository upload steps.

The most important coding decision was to keep the project understandable. Using vanilla JavaScript allowed the project to show direct knowledge of DOM events, forms, functions, async/await, arrays, tables, and API calls. Using Supabase prevented the need to write a full custom back-end server, which kept the project realistic for a college-level web development assignment.

2.3 Architecture

The application follows a simple static-front-end plus cloud-database architecture. The front-end files are hosted on Cloudflare Pages. The browser downloads HTML, CSS, and JavaScript. JavaScript communicates with Supabase for

authentication, database tables, and SQL functions. GitHub stores the code repository and Cloudflare Pages deploys from it.

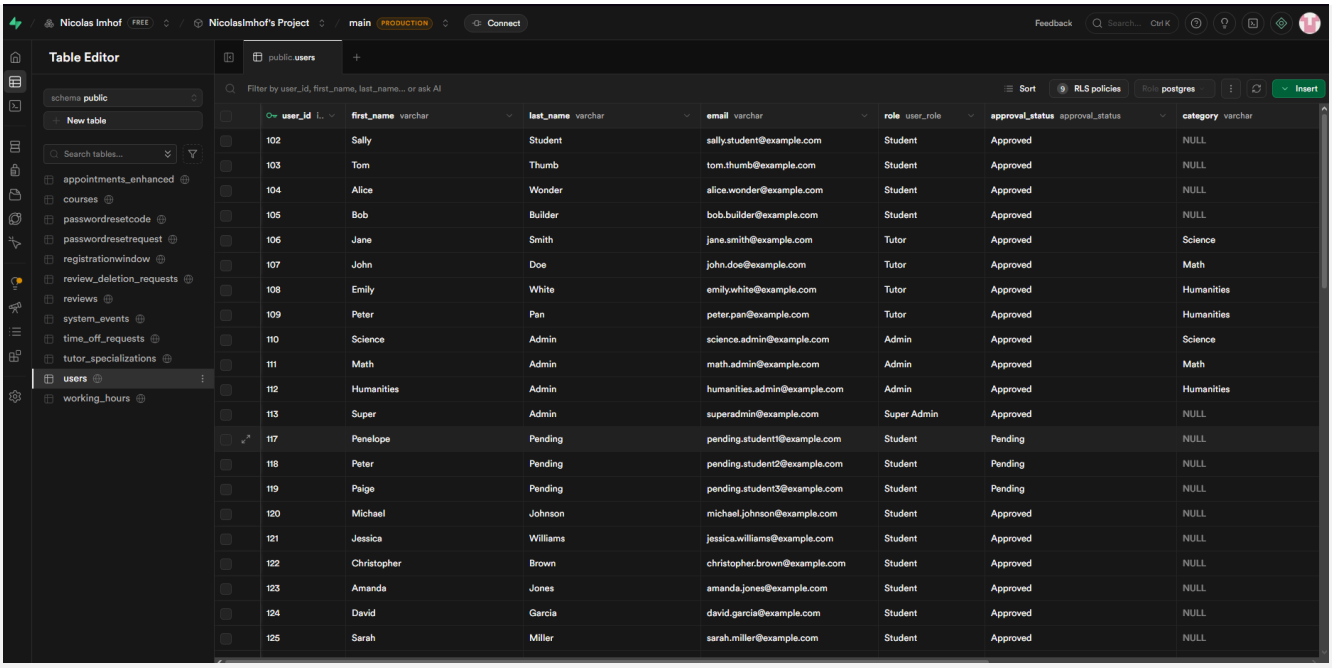
- Client layer: index.html, register.html, style.css, calendar.css, script.js, and calendar.js.
- Authentication layer: Supabase Auth handles sign-up, login, sessions, and password updates.
- Data layer: Supabase PostgreSQL stores users, courses, appointments, reviews, time-off requests, working hours, and system events.
- Business logic layer: SQL/RPC functions handle tutor lists, calendar events, appointment booking, and availability slots.
- Hosting layer: Cloudflare Pages serves the static front-end from the GitHub repository.

2.4 Database Tables and Fields

The system uses relational data because users, tutors, courses, schedules, appointments, and reviews all connect to one another. The database structure is handled through SQL migrations and seed files in the project. The following table summarizes the most important database tables used by the application.

Table	Purpose
users	Stores profile records for students, tutors, admins, and super admins. Includes role, approval status, category, major, email, and auth UUID.
courses	Stores course names and course codes used to describe what tutors can teach.
tutor_specializations	Join table that links tutors to the courses they specialize in.
appointments_enhanced	Stores tutor/student appointments with start time, end time, status, and optional course.
working_hours	Stores a tutor weekly schedule by day of week, start time, end time, and whether that day is active.
time_off_requests	Stores tutor time-off requests and status values such as Pending or Approved.
system_events	Stores school-wide or category events such as holidays and closures. These can block appointment booking.
reviews	Stores student reviews, ratings, tutor responses, and deletion request status.

The seed.sql file contains sample students, tutors, admins, courses, working hours, appointments, reviews, and a pending time-off request. This seed data was made so that every major screen can be demonstrated without manually creating dozens of records first.



2.5 API/RPC Development

Instead of creating a separate Express API server, the project uses Supabase RPC functions. This method was chosen because it keeps database-related business rules close to the database. For example, appointment validation belongs in a database function because it must check working hours, existing appointments, time-off requests, and holidays before a booking is created.

Function	Purpose
get_student_tutor_list()	Returns approved tutors with category, average rating, review count, and specializations for the Find a Tutor page.
get_tutor_availability_slots(tutor_id, date)	Generates available 30-minute slots by checking working hours, appointments, approved time off, and system events.
book_appointment(student_id, tutor_id, start, end)	Creates an appointment only after checking working hours, appointment conflicts, time-off conflicts, and system event conflicts.
get_appointments_for_calendar(...)	Returns calendar appointments based on the logged-in user role and category.
get_calendar_events(...)	Returns visible system events based on date range, role, and category.
create_system_event(...)	Allows a Super Admin to create holiday or closure events that appear on the calendar.

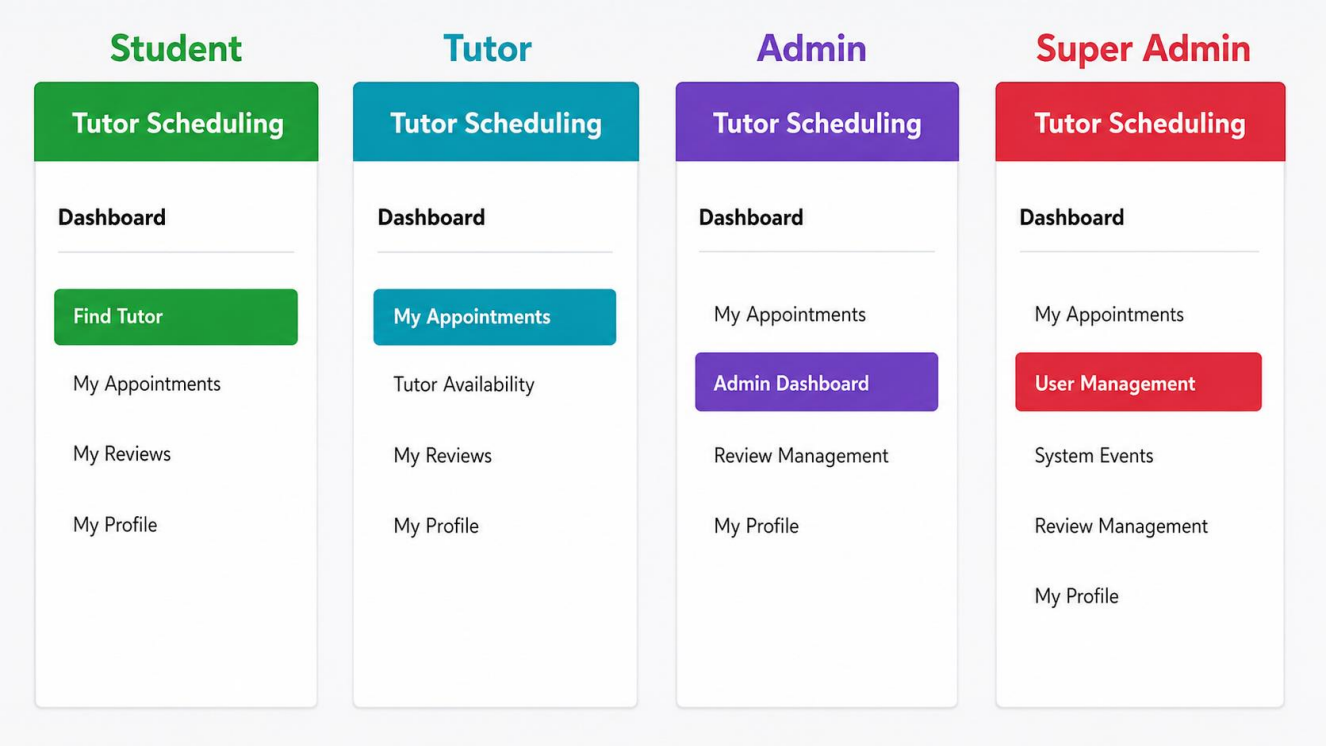
The strongest example is book_appointment. That function does not simply insert an appointment. It validates the requested time, checks whether the tutor works that day, checks for appointment overlaps, checks approved time off, checks system events, and only then inserts the appointment.

```
const { data, error } = await this.supabase.rpc('book_appointment', {
    p_student_id: this.currentUser.user_id,
    p_tutor_id: tutor.user_id,
    p_start_time: startTime.toISOString(),
    p_end_time: endTime.toISOString()
});
```

2.6 System Organization and Navigation

After login, the dashboard is rendered dynamically by script.js. The navigation links are generated from the viewPermissions object, which maps each screen to the roles allowed to access it. This prevents every role from seeing every tab and keeps each dashboard focused.

Role	Navigation Tabs	Main Functions
Student	My Profile, Find a Tutor, My Appointments, My Reviews	Find tutors, search by name/category, review tutor ratings, book appointments, view/cancel appointments, write/edit reviews.
Tutor	My Profile, My Availability, My Appointments, My Reviews	Manage time-off requests, view scheduled sessions, view anonymous student reviews, respond to reviews, request review deletion.
Admin	My Profile, Admin Dashboard, My Appointments, Review Management	Oversee tutors in the admin category, approve tutor time-off requests, view category appointments/reviews, manage review deletion requests.
Super Admin	My Profile, User Management, System Events, Review Management, My Appointments	Approve new accounts, delete users, create system-wide events, process review deletion requests, view full system activity.



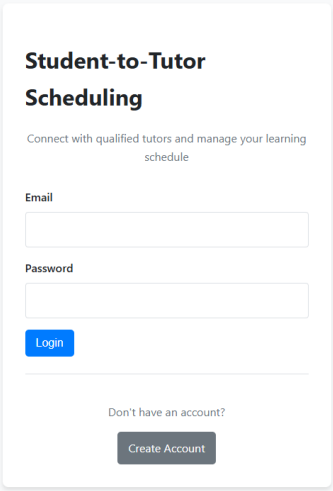
3. Using the System

3.1 Login Page

The login page is the default page at the deployed website. The user enters an email and password. JavaScript calls Supabase authentication through signInWithPassword. If authentication succeeds, the system fetches the user profile from the users table. If the profile approval_status is not Approved, the system logs the user back out and tells the user the account is pending approval.

- Expected input: email address and password.

- Expected output: successful dashboard login for approved users, or an error/pending message for invalid or unapproved users.
- Possible failure: wrong password, missing auth user, unapproved account, or broken Supabase connection.

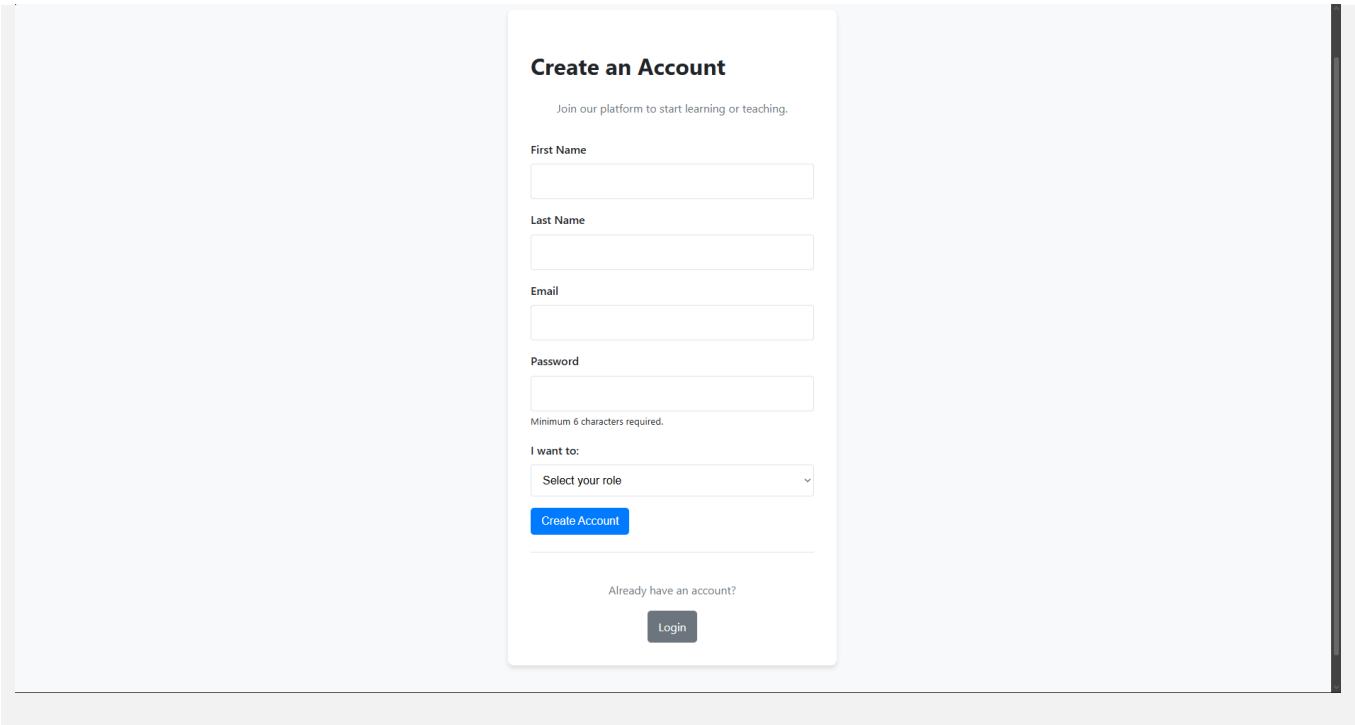



```
async login() {
    const email = document.getElementById('email').value;
    const password = document.getElementById('password').value;
    const { data, error } = await this.supabase.auth.signInWithPassword({ email, password });
    if (error) alert(`Login failed: ${error.message}`);
    else if (data.user) this.checkSession();
}
```

3.2 Registration Page

The registration page is reached by clicking Create Account from the login page. A new user provides first name, last name, email, password, and whether they want to be a Student or Tutor. The page uses Supabase signUp and passes user metadata for first name, last name, and role. New accounts are created with Pending approval, so a Super Admin must approve them before login is allowed.

- Expected input: first name, last name, email, password with at least 6 characters, and selected role.
- Expected output: success message explaining that the account is pending approval.
- Possible failure: email already exists, password too short, missing field, or Supabase sign-up issue.



3.3 Dashboard Shell

The dashboard shell contains the top header, user name, logout link, sidebar navigation, content area, and footer. The dashboard does not exist as separate hard-coded HTML pages. Instead, it is generated by script.js after the user session and profile are confirmed. This approach avoids copying the same header and sidebar into multiple files.

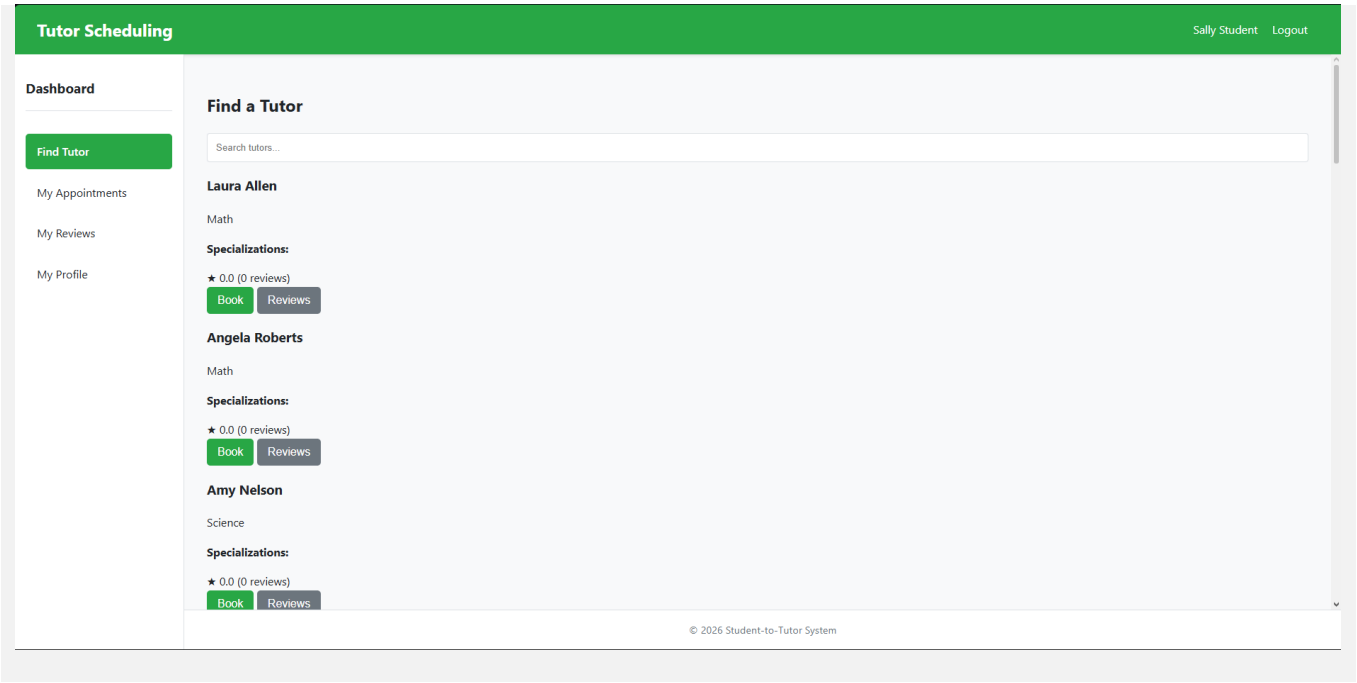
The loadView(view) method behaves like a client-side router. When a sidebar link is clicked, the method empties and repopulates the dashboard content area with the selected feature.

```
loadView(view) {
    const content = document.getElementById('dashboard-content');
    switch (view) {
        case 'find-tutor': this.renderFindTutor(content); break;
        case 'my-appointments': this.renderMyAppointments(content); break;
        case 'admin-dashboard': this.renderAdminDashboard(content); break;
        case 'user-management': this.renderUserManagement(content); break;
    }
}
```

3.4 Student: Find a Tutor

The Find a Tutor page is the default student page. It loads approved tutors through the get_student_tutor_list RPC function. Each tutor card shows the tutor name, category, specializations, average rating, review count, Book button, and Reviews button. The search box filters the visible tutor cards by name or category.

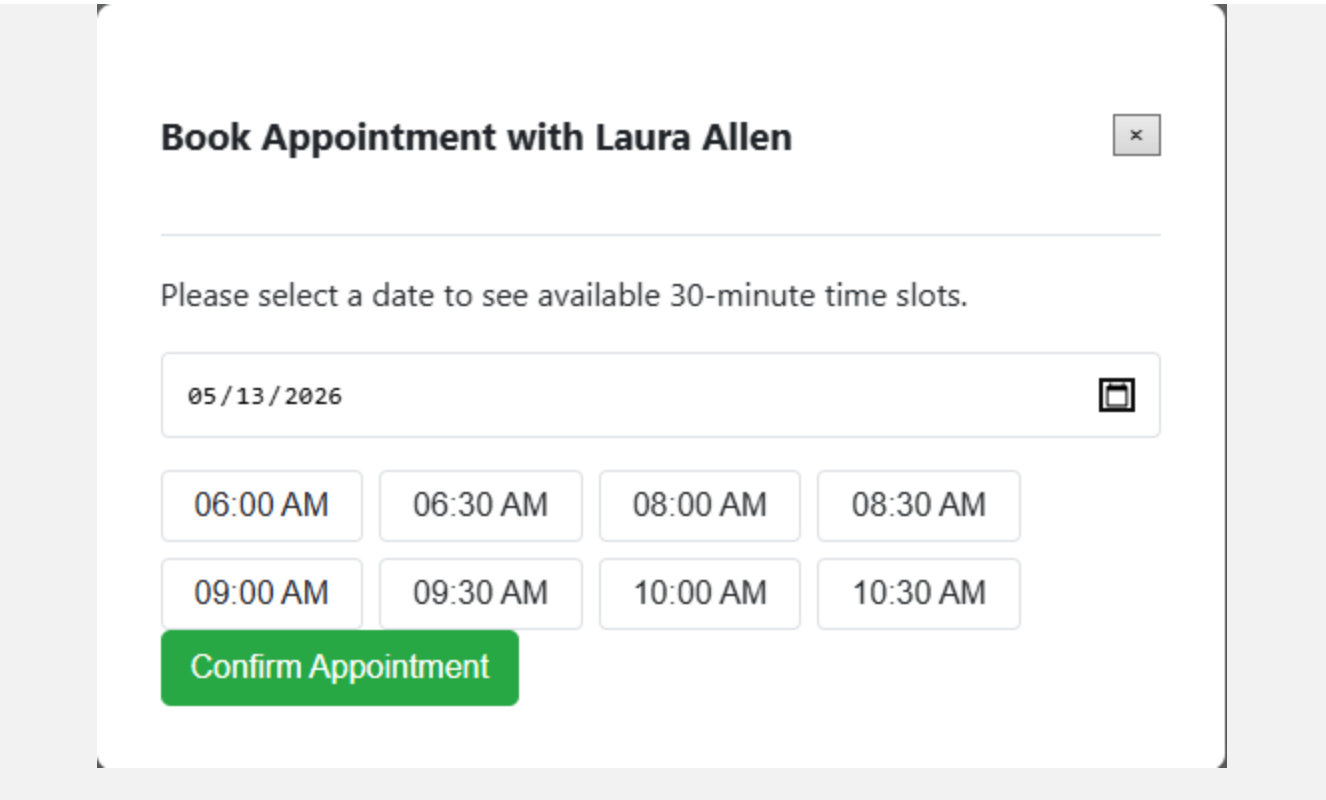
- Book button: opens the appointment booking modal for the selected tutor.
- Reviews button: opens a modal showing reviews for the selected tutor.
- Search bar: filters tutor cards on the page without requiring a new database call.



3.5 Student: Booking Appointments

When a student clicks Book, the system opens a modal. The student selects a date. The system calls get_tutor_availability_slots to display only valid 30-minute time slots. After selecting a slot, the Confirm Appointment button becomes active. The actual booking is handled by the book_appointment RPC function so the database can protect against conflicts.

37. Student clicks Book on a tutor card.
38. Student chooses a date.
39. System loads available 30-minute slots for that tutor and date.
40. Student clicks a slot.
41. Student clicks Confirm Appointment.
42. Supabase checks working hours, appointment conflicts, time off, and system events.
43. If all checks pass, appointment is created and the student is redirected to My Appointments.

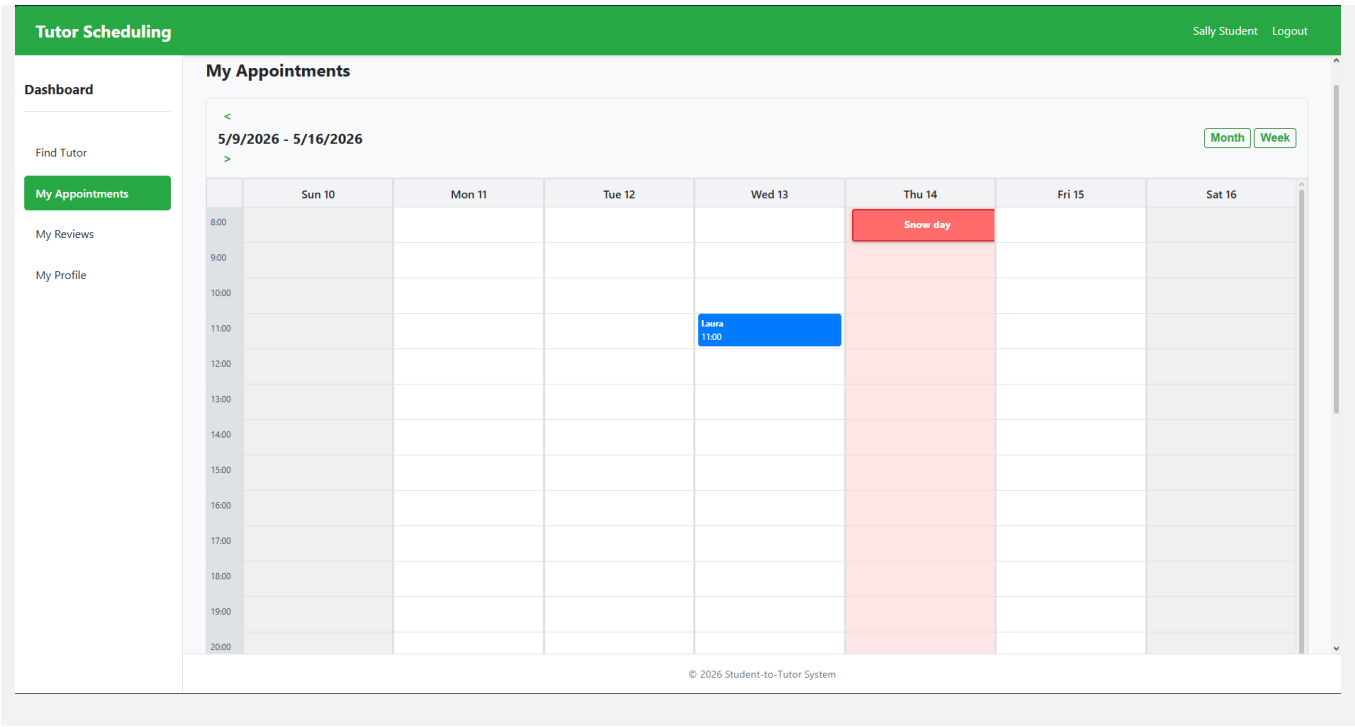


3.6 My Appointments and Calendar

The My Appointments page combines a calendar view with an appointment list. The Calendar module supports week and month views. Students see their own appointments, tutors see appointments where they are the tutor, admins see appointments within their category, and super admins can view broader system appointments.

The calendar was separated into calendar.js and calendar.css because calendar display logic is large enough to be its own module. This kept script.js focused on account, dashboard, and feature rendering.

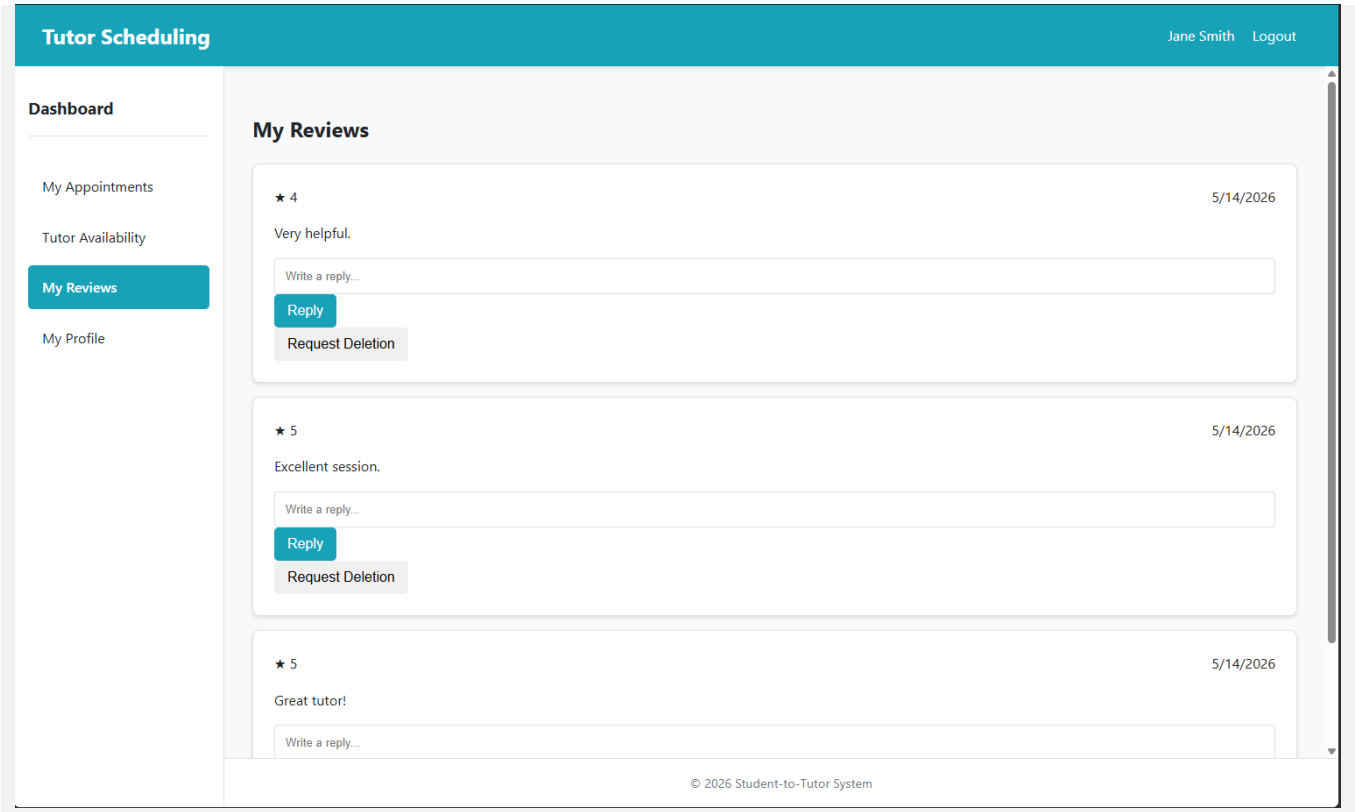
- Week view: shows a time grid and event blocks.
- Month view: shows a month grid and event summaries.
- Navigation buttons: move the view backward or forward.
- Month/Week toggle: switches between broad and detailed calendar views.
- Cancel appointment button: marks a Scheduled appointment as Cancelled.



3.7 Reviews

The reviews system gives students, tutors, admins, and super admins different levels of access. A student can view reviews they wrote. A tutor can view reviews about them anonymously and respond once. Tutors can also request deletion if a review should be reviewed by an administrator. Admins and super admins can view deletion requests in Review Management.

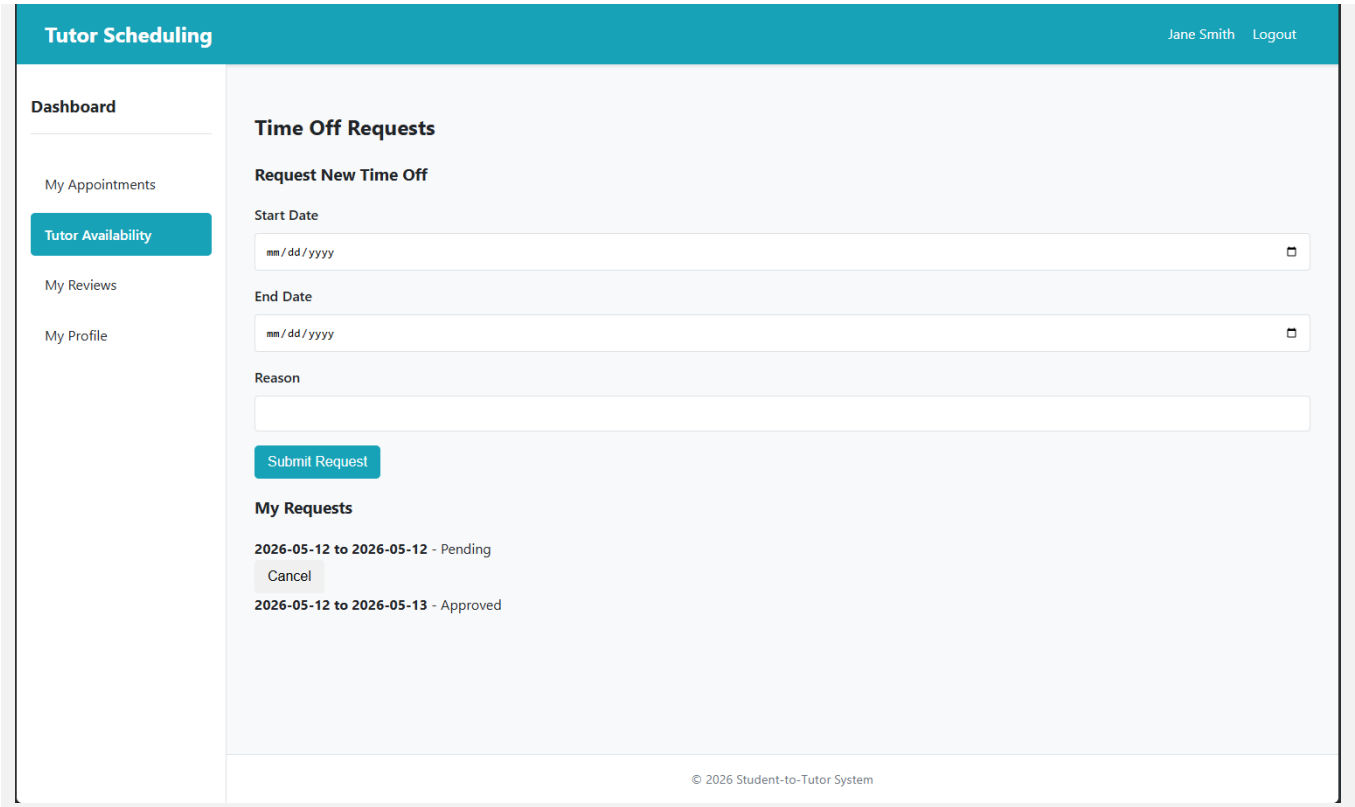
- Student view: shows reviews connected to the student.
- Tutor view: hides the student identity and allows tutor response.
- Admin view: shows deletion requests for tutors in the admin category.
- Super Admin view: can process deletion requests across the system.



3.8 Tutor: My Availability / Time Off

Tutors use the availability page to submit time-off requests. The tutor enters a start date, end date, and reason. The request is saved as Pending. The tutor can cancel a pending request before it is approved. Approved time off blocks availability during appointment booking.

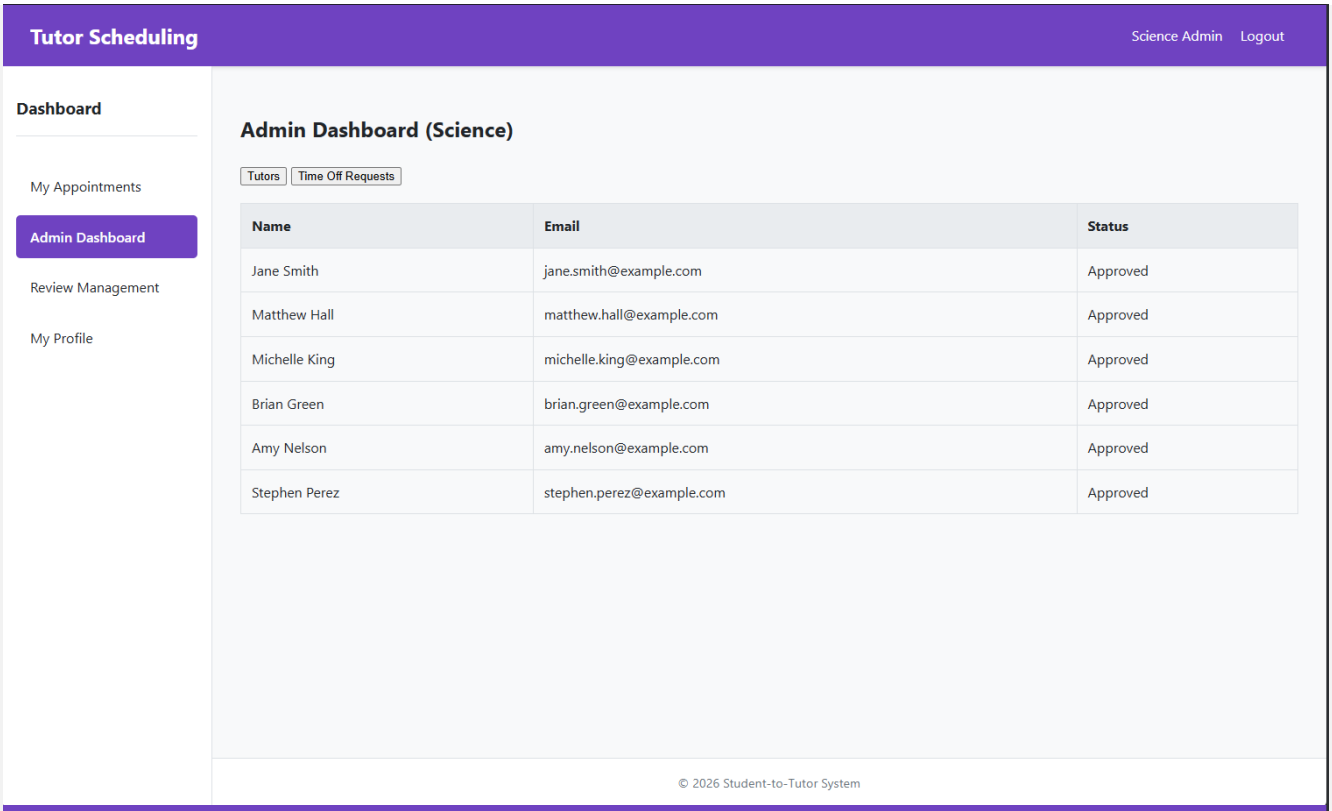
44. Tutor opens My Availability.
45. Tutor enters start date, end date, and reason.
46. Tutor submits the request.
47. Request appears in My Requests as Pending.
48. An admin in the tutor category approves it from Admin Dashboard.
49. After approval, booking logic prevents students from booking during that date range.

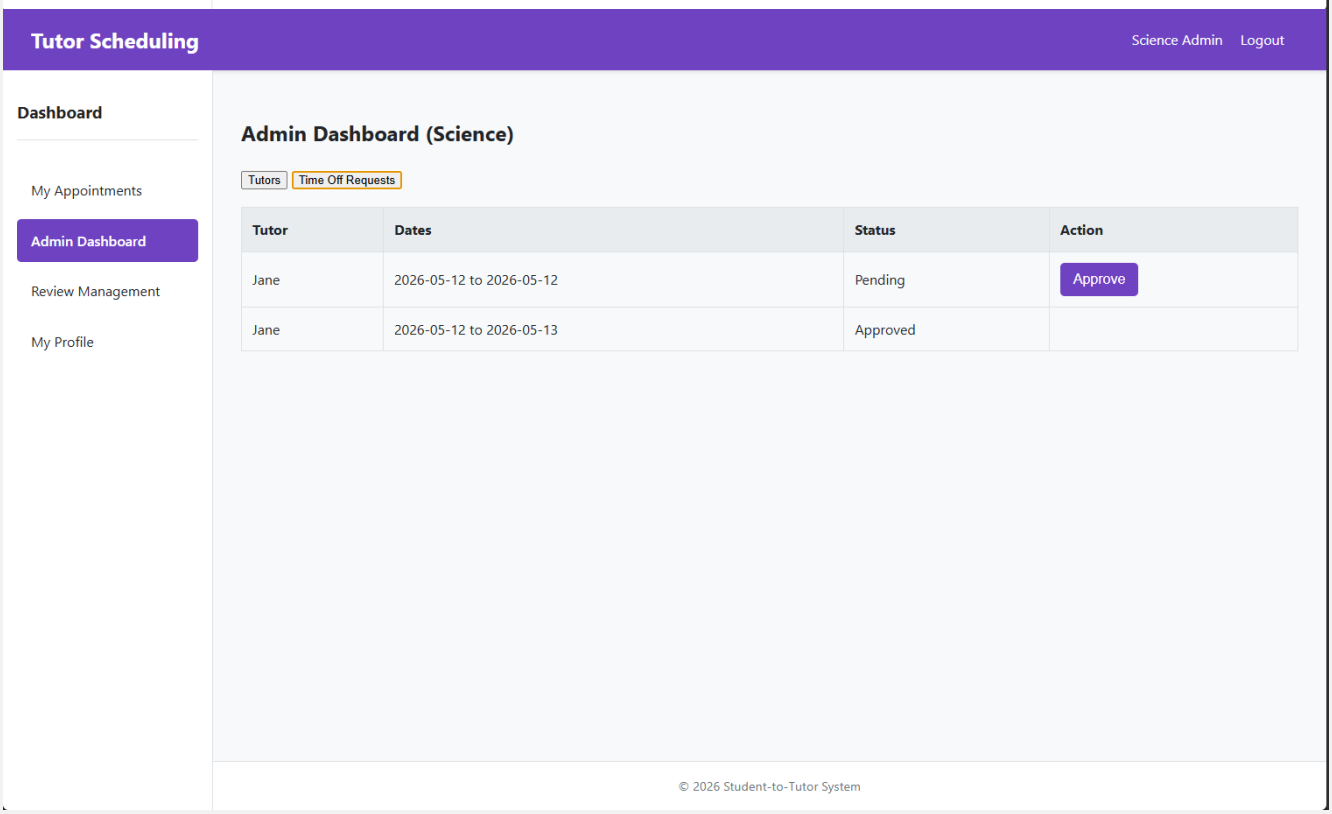


3.9 Admin Dashboard

The Admin Dashboard is category-specific. For example, a Science admin only oversees Science tutors. The dashboard has tabs for Tutors and Time Off Requests. The Tutors tab lists category tutors and approval statuses. The Time Off Requests tab lets the admin approve pending requests for tutors in their category.

- Restriction: Admins cannot create or delete user accounts.
- Jurisdiction: Admin views are filtered by currentUser.category.
- Main output: tables showing tutors and pending time-off requests.

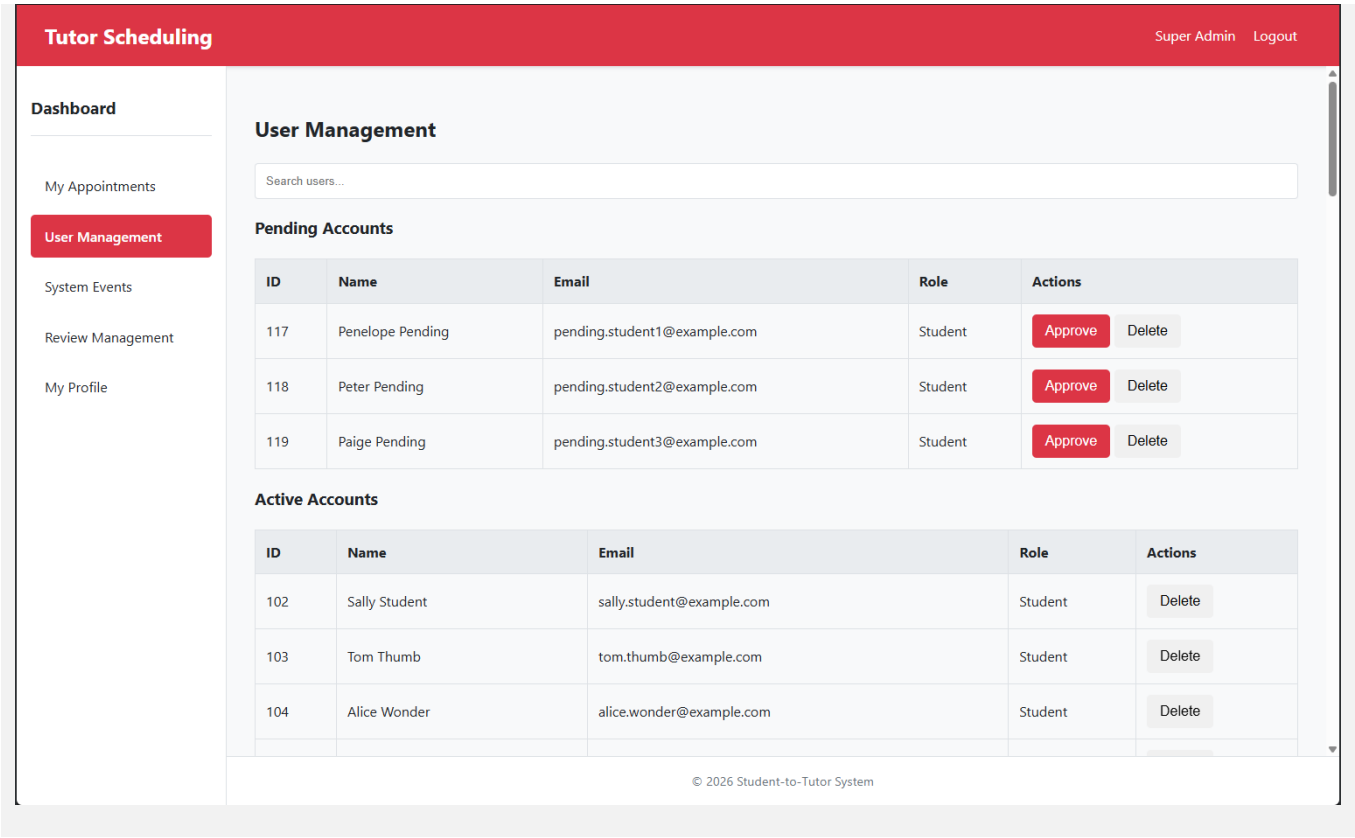

Tutor Scheduling
Science Admin
Logout
Dashboard
My Appointments
Admin Dashboard
Review Management
My Profile
Admin Dashboard (Science)
Tutors
Time Off Requests
Name
Email
Status
Jane Smith
jane.smith@example.com
Approved
Matthew Hall
matthew.hall@example.com
Approved
Michelle King
michelle.king@example.com
Approved
Brian Green
brian.green@example.com
Approved
Amy Nelson
amy.nelson@example.com
Approved
Stephen Perez
stephen.perez@example.com
Approved
© 2026 Student-to-Tutor System


Tutor Scheduling
Science Admin
Logout
Dashboard
My Appointments
Admin Dashboard
Review Management
My Profile
Admin Dashboard (Science)
Tutors
Time Off Requests
Tutor
Dates
Status
Action
Jane
2026-05-12 to 2026-05-12
Pending
Approve
Jane
2026-05-12 to 2026-05-13
Approved
© 2026 Student-to-Tutor System

3.10 Super Admin User Management

Super Admins use User Management to view accounts, approve pending accounts, and delete users. This is required because new Student and Tutor registrations are not automatically approved. Account approval provides a control point so users cannot immediately access the system without administrative review.

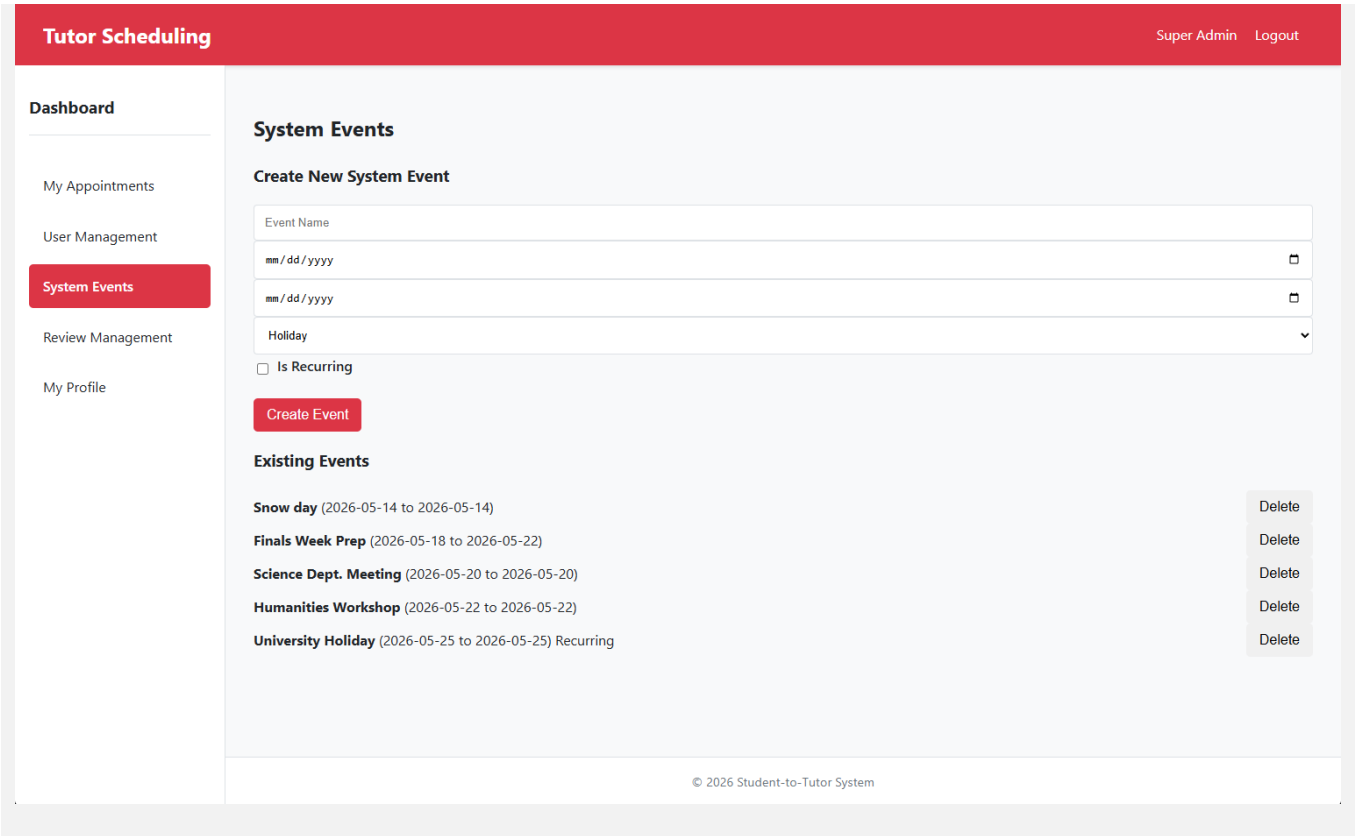
- Approve button: changes approval_status from Pending to Approved.
- Delete button: removes a user record. Related data should be protected by foreign keys and cascade behavior where configured.
- Purpose: keep account control centralized under the Super Admin.



3.11 System Events

The System Events page lets a Super Admin create holidays or closures. A system event has a name, start date, end date, and event type. These events appear in calendar views and are checked by appointment booking functions so students cannot book during system-wide closures.

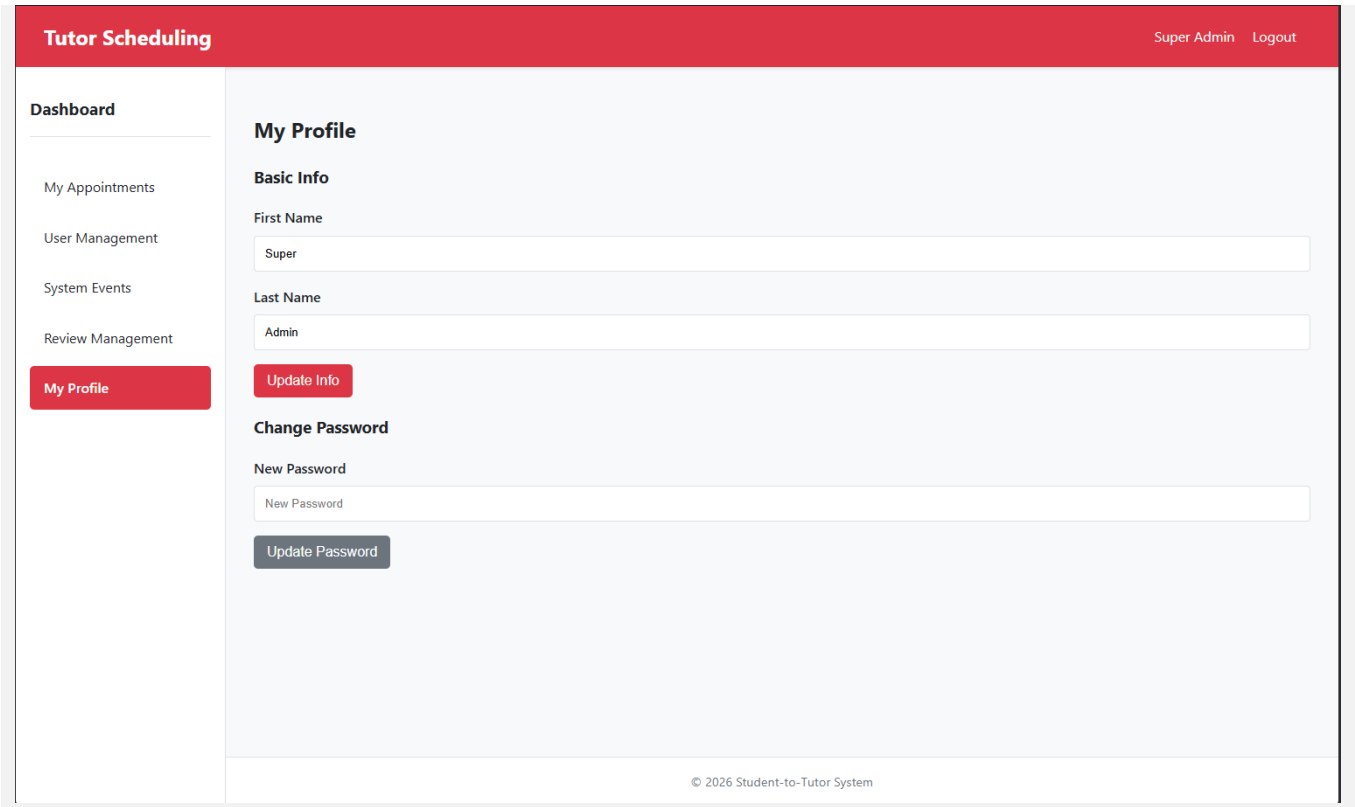
- Example event: Spring Break, Final Exam Week, Campus Closure, Holiday.
- Effect: event becomes visible on the calendar and can block booking.
- Future improvement: add recurring event editing and event deletion controls from the dashboard.



3.12 My Profile

The My Profile page lets the logged-in user update first name, last name, and password. Updating profile information changes the public users table. Updating the password uses Supabase Auth updateUser so the authentication password is changed securely through Supabase.

- Basic Info form: first name and last name.
- Change Password form: new password.
- Output: alert message confirming update or showing the Supabase error.



3.13 Logout

The logout link signs out the current Supabase session, clears the currentUser value, empties the dashboard container, resets the body class, and returns the user to the login view. This prevents someone from using dashboard features after a logout.

```
async logout() {
    await this.supabase.auth.signOut();
    this.currentUser = null;
    document.getElementById('dashboard-container').innerHTML = '';
    document.getElementById('login-view').classList.remove('hidden');
}
```

4. Local Installation and Execution

4.1 Empty-Computer Setup

This section assumes a person is starting from a computer that does not already have any project tools installed. The goal is that the person can install the system without guessing.

50. Install Trae AI IDE from the official Trae download page. Choose TRAE IDE and Windows x64 if using Windows.
51. Install Node.js LTS from the official Node.js website. npm installs with Node.js.
52. Install Git from git-scm.com if using GitHub from the terminal.
53. Download and extract the project zip. Do not work from inside the compressed zip.
54. Open Trae AI and open the extracted project folder.
55. Open the integrated terminal and confirm the project root contains package.json.

56. Run node -v, npm -v, and git --version to verify the tools are detected.
57. Run npm install to download @supabase/supabase-js, http-server, and any dependency tree listed in package-lock.json.
58. Create or confirm the Supabase project. Copy the project URL and anon key into script.js if using a different Supabase instance.
59. Run the SQL migrations and seed.sql in Supabase SQL Editor if recreating the database.
60. Run npm start and open http://localhost:8000.

4.2 Supabase Database Setup

Supabase is a cloud-hosted backend. There is no desktop Supabase program required for this workflow. The database is managed through the Supabase dashboard and SQL editor.

61. Create a Supabase account and open the dashboard.
62. Create a new project. Save the project URL and public anon key from Project Settings > API.
63. Open SQL Editor.
64. Run the migration files in logical order. The migrations folder contains early calendar/tutor management migrations and the supabase/migrations folder contains later fixes and RPC updates.
65. Run seed.sql to insert test courses, users, tutor specializations, working hours, time-off requests, appointments, and reviews.
66. Create Supabase Auth users for the demo accounts if needed. The seed_auth_users.js helper should use an environment variable for SUPABASE_SERVICE_KEY rather than hard-coding the secret key.
67. Check the users table to confirm each test account has an auth_uuid if auth login will be tested locally.

Important security note: the service role key should never be placed in a public GitHub repository or public document. The anon key can be used in browser code, but the service role key has admin privileges and should stay private.

4.3 Running Locally

```
cd "STUDENT-TO-TUTOR SCHEDULING AND AVAILABILITY MANAGEMENT SYSTEM"
npm install
npm start
```

After npm start, open the local URL shown by the terminal, usually http://localhost:8000. Test the login page, registration page, each role dashboard, appointment booking, reviews, profile page, and logout.

4.4 Test Accounts

The seeded database includes demo accounts. In the project context, the common test password is password. These accounts are for demonstration only and should not be used in production.

Role	Email	Purpose
Student	sally.student@example.com	Book tutors, view student appointments, view student reviews.
Tutor	jane.smith@example.com	View tutor appointments, submit time off, respond to reviews.
Admin	science.admin@example.com	Approve Science tutor time off and review deletion requests.
Super Admin	superadmin@example.com	Approve users, manage system events, review deletion requests.

5. System Performance

5.1 Testing Performed

Testing was performed through browser-based feature testing. The goal was to make sure each role could access only the correct dashboard pages and that major actions changed the database as expected.

- Login and logout were tested with multiple roles.
- Registration was tested to confirm new accounts become Pending.
- Find a Tutor was tested to confirm tutor cards, search filtering, reviews, and booking modals load.
- Appointment booking was tested against tutor availability and conflict checks.
- Admin time-off approval was tested by approving pending requests.
- Super Admin user approval and system event creation were tested.
- Cloudflare Pages deployment was tested by visiting the final URL.

5.2 State of the System as Delivered

As delivered, the system contains a functional static front end, Supabase database integration, role-based navigation, authentication, tutor search, appointment booking, calendar display, review handling, time-off requests, user approval, and system event creation. The deployed website is available at the Cloudflare Pages URL listed on the title page.

5.3 Known Limitations

- The project is an academic prototype and should not be considered production-ready without deeper security review.
- Some administrative workflows are simplified, such as limited editing controls after a system event is created.
- The tutor weekly availability editor is not fully expanded in the UI; working hours are mainly handled through seeded/database records.
- The app uses a static front-end and Supabase direct calls, so database policies must be strong before real users are added.
- The project currently uses a free pages.dev domain instead of a paid custom domain.
- The UI focuses on core functionality and may need more accessibility testing before institutional use.

5.4 Future Improvements

Priority	Feature	Reason
High	Email notifications	Send confirmation, cancellation, and approval emails for bookings, reviews, and time-off requests.
High	Tutor availability editor	Add a full weekly hour editor in the tutor dashboard instead of relying mostly on seed/database working-hour records.
High	Stronger row-level security	Add stricter Supabase policies for every table so database rules protect data even if UI code is changed.
Medium	Review editing workflow	Allow students to edit reviews from a clean form and let tutors see a clearer response history.
Medium	Notifications dashboard	Show pending approvals, new reviews, cancellations, and time-off status changes.
Medium	Course filtering	Add filters for course, subject, rating, and tutor availability.
Low	Calendar export	Allow export to Google Calendar or Outlook using ICS files.
Low	Paid/custom domain	Replace the pages.dev subdomain with a custom domain if the project becomes a real service.

6. Troubleshooting and Support

6.1 Error Messages

Error / Symptom	Likely Cause	Corrective Action
Login failed	Wrong email/password, missing verified Supabase auth user, or browser cache issue.	Check email/password, confirm the user exists in Supabase Authentication, approve the user, then retry.
Your account is pending approval	The user exists but approval_status is not Approved.	Log in as Super Admin and approve the account from User Management.
RPC Error on Find a Tutor	The required Supabase function is missing or an old migration was not applied.	Apply the latest migration files in Supabase SQL Editor, especially master RPC fixes.
Could not load availability	Availability RPC cannot return slots, the tutor has no working hours, or the database function is missing.	Check working_hours for the tutor and run get_tutor_availability_slots migration.
Tutor is not available at this time	The selected slot is outside working hours or blocked.	Choose another day/time or update tutor working hours.
Appointment already exists at this time	Conflict with an existing scheduled appointment.	Pick another time slot.
System-wide holiday or closure	The requested slot falls during a system_event.	Choose a date outside the holiday/closure.
npm is not recognized	Node.js/npm is not installed or terminal was opened before installation completed.	Install Node.js LTS, close Trae, reopen Trae, and run node -v and npm -v again.

6.2 Special Considerations

- Keep the service role key private. Do not commit .env files or service keys to GitHub.
- Run npm install after extracting the project. Do not manually copy node_modules unless absolutely necessary.
- Use the latest migration files. Older SQL functions may conflict with newer function signatures.
- If Supabase functions appear cached or outdated, drop the old function signature and recreate the latest one from the migration file.
- When debugging in Trae AI, use the browser console and Trae terminal together. Browser console errors usually point to front-end or Supabase request problems.

6.3 Support

Primary project contact: Nicolas Imhof, nimhof@oldwestbury.edu. Because this is an individual academic project, support is provided by the project creator for course review and demonstration purposes.

If a problem occurs during grading or demonstration, the recommended first check is the deployed Cloudflare Pages URL, then Supabase project status, then browser console errors, then local npm installation.

Appendix A: References

Reference	Location
Trae AI Download	https://www.trae.ai/download
Trae AI Quickstart	https://docs.trae.ai/ide/set-up-trae?_lang=en
npm Docs - Downloading and installing Node.js and npm	https://docs.npmjs.com/downloading-and-installing-node-js-and-npm/
Supabase JavaScript Installing	https://supabase.com/docs/reference/javascript/installing

Supabase JavaScript Introduction	https://supabase.com/docs/reference/javascript/introduction
GitHub Docs - Adding locally hosted code to GitHub	https://docs.github.com/en/migrations/importing-source-code/using-the-command-line-to-import-source-code/adding-locally-hosted-code-to-github
Cloudflare Pages - Git integration	https://developers.cloudflare.com/pages/get-started/git-integration/
Cloudflare Pages - Direct Upload	https://developers.cloudflare.com/pages/get-started/direct-upload/

Appendix B: Key Terms

Term	Definition
Authentication	The process of verifying that a user is allowed to sign in with an email/password account.
Authorization	The process of determining which role-based pages and actions the signed-in user is allowed to use.
Supabase RPC	A database function called remotely from JavaScript using supabase.rpc().
RLS	Row Level Security; Supabase/PostgreSQL rules that control which rows a user can select, insert, update, or delete.
Static hosting	Hosting method where HTML, CSS, and JavaScript files are served directly without a custom server process.
pages.dev	The free Cloudflare Pages domain suffix used for the deployed website.
Seed data	Sample data inserted into the database for testing and demonstration.

Appendix C: Important Code Segments

C.1 Supabase Client Initialization

```
const supabaseUrl = 'https://bycjjodkedhynxkckwtg.supabase.co';
const supabaseAnonKey = 'public-anon-key';
this.supabase = supabase.createClient(supabaseUrl, supabaseAnonKey);
```

C.2 Role-Based Permissions Object

```
viewPermissions: {
    'find-tutor': ['Student'],
    'my-appointments': ['Student', 'Tutor', 'Admin', 'Super Admin'],
    'tutor-availability': ['Tutor'],
    'my-reviews': ['Student', 'Tutor'],
    'admin-dashboard': ['Admin'],
    'user-management': ['Super Admin'],
    'system-events': ['Super Admin'],
    'review-management': ['Admin', 'Super Admin'],
    'my-profile': ['Student', 'Tutor', 'Admin', 'Super Admin'],
}
```

C.3 Local npm Commands

```
npm install
npm start
```